

임장혁

Backend Developer

Java/Spring Boot와 TypeScript/NestJS 기반으로 상태 변화, 비동기 흐름, 실시간 통신을 다루는 백엔드 시스템을 구현해 왔습니다.

상태 변화와 비동기 흐름이 많은 서비스를 안정적으로 다루는 백엔드 개발자를 지향합니다. 기능이 동작하는지에서 멈추지 않고, 실패 조건과 사용자 체감 문제를 코드와 테스트로 확인하려고 합니다.

연락처

Email: qkdrhkg23@naver.com

LinkedIn: linkedin.com/in/imjanghyeok

링크

GitHub: github.com/imjanghyeok

Blog: dlaljh.tistory.com

Portfolio PDF: website/assets/portfolio.pdf

주요 기술

Java, Spring Boot, NestJS, Node.js, TypeScript

MySQL, PostgreSQL, MongoDB, JPA, TypeORM

중점 경험

Kafka, Redis, WebSocket/STOMP

Docker, GitHub Actions, E2E Test

프로필 요약 · 핵심 역량 · 경력 및 활동 이력

저는 상태 변화와 비동기 흐름이 많은 서비스를 안정적으로 다루는 백엔드 개발자를 지향합니다. AI 면접 서비스의 세션 흐름, MSA 메시저의 메시지 저장/조회 정합성, WebSocket 채팅의 접속 상태 관리처럼 사용자가 기능 성공 여부를 직접 체감하는 문제를 다뤘습니다. 단순 구현보다 실패 조건, 데이터 정합성, 테스트 가능성을 함께 확인하는 방식으로 일합니다. 서비스 흐름을 안정적으로 설계하고, 변경이 누적되어도 검증 가능한 백엔드 구조를 만드는 데 기여할 수 있습니다.

핵심 기술 요약

Backend Java, Spring Boot, NestJS, Node.js, TypeScript	Data MySQL, PostgreSQL, MongoDB, JPA, TypeORM	Infra Docker, Docker Compose, GitHub Actions, AWS, NCP, Nginx
--	---	---

핵심 강점

- AI 면접 서비스에서 LLM 응답을 서버 정책으로 제어하고, 질문 중복 방지와 세션 만료 흐름을 구현했습니다.
- MSA 메시저에서 Kafka 저장 지연이 사용자에게 메시지 유실처럼 보이지 않도록 조회 누락 가능성을 완화하는 경로를 구성했습니다.
- WebSocket/STOMP 채팅에서 접속 상태와 채팅방 상태를 Redis에 분리해 저장했습니다.
- 객체지향 설계 기반 키오스크 시스템에서 도메인 모델링과 계층 분리를 적용했습니다.

업무 역량

상태 모델링 영속 데이터, 런타임 상태, 현재성 정보를 구분하고 저장 위치를 선택합니다.	검증 가능한 구현 실패 조건을 테스트와 CI 흐름으로 옮겨 변경 안정성을 높입니다.
비동기 흐름 Kafka, WebSocket, polling 흐름에서 사용자가 체감하는 정합성을 봅니다.	협업 문서화 왜 이 구조를 선택했는지 팀원이 따라올 수 있는 형태로 남깁니다.

경력 및 활동 이력 요약

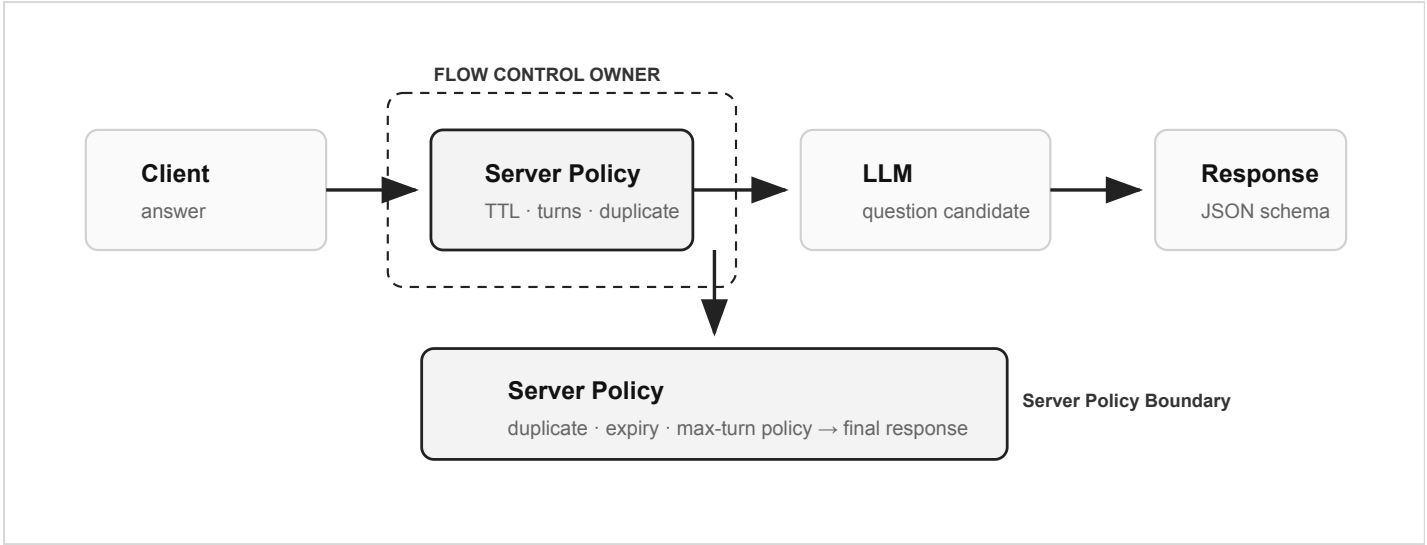
2025.01.05 ~ 2026.02.06 · Synapse AI 면접 시뮬레이션 서비스 · Backend Developer
2025.01 ~ 2025.03 · SmileTogether 실시간 메신저 플랫폼 · Team Lead & Backend Developer
2024.03 ~ 2024.06 · FaceFriend AI 이미지 분석 기반 데이팅 커뮤니티 · Backend Developer
2024.03 ~ 2024.06 · KiKi Kiosk 객체지향 기반 키오스크 시스템 · Backend Developer
2023.06 ~ 2023.07 · Relanz 스트레스 관리 및 챌린지 플랫폼 · Backend/Web Developer

5. 주요 프로젝트 / 작업물 · Synapse

AI 면접 시뮬레이션 서비스 · 2025.01.05 ~ 2026.02.06 · TypeScript, Node.js, NestJS, MySQL, TypeORM, Docker

역할 Backend Developer	핵심 문제 LLM 비결정성과 면접 흐름 제어	검증 Unit, E2E, CI 단계 보강
--------------------------------	------------------------------------	----------------------------------

Synapse는 사용자 문서 기반으로 AI 면접 질문을 생성하는 서비스입니다. 문제는 LLM 응답이 항상 서비스 정책을 지키지 않는다는 점이었습니다. 질문이 반복되거나, 종료 조건이 늦게 반영되거나, 면접이 의도보다 길어지는 흐름은 사용자가 서비스 품질 문제로 체감할 수 있습니다.



제가 맡은 일

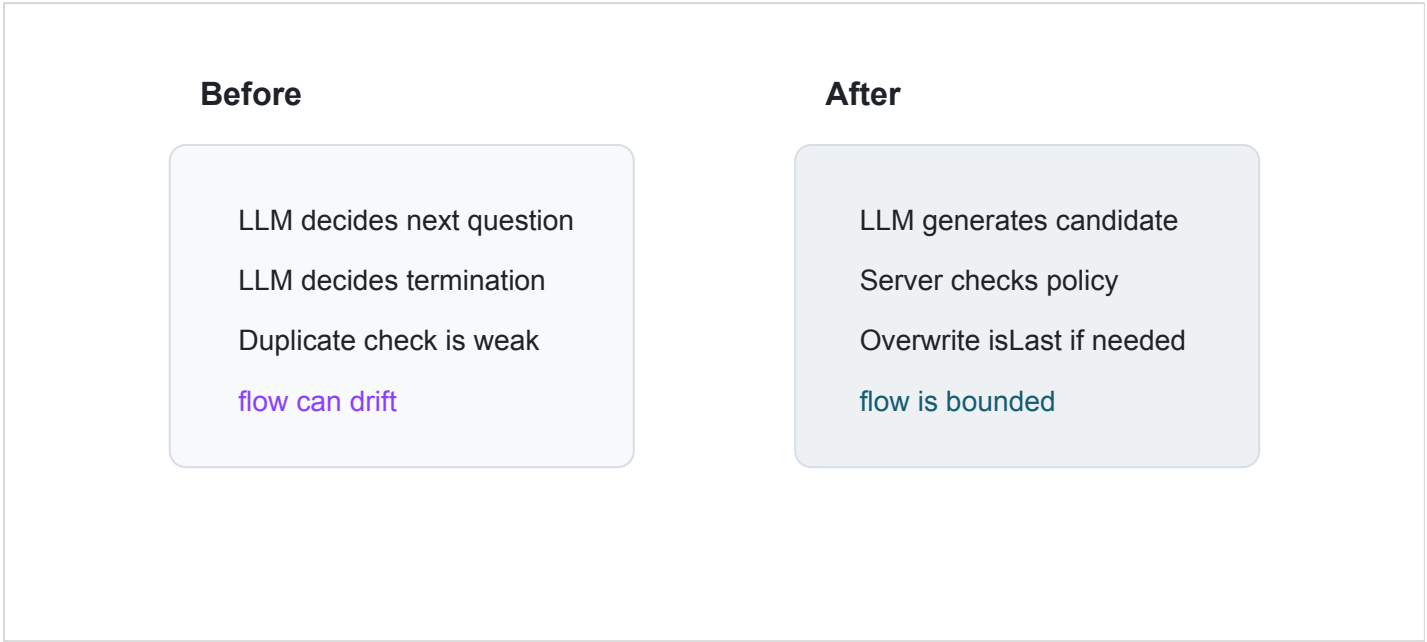
- 서버 정책 기반 질문 흐름 제어 모듈 구현
- 질문 생성 로직의 사용자 문서 컨텍스트 반영, 중복 방지, 재시도 흐름 보강
- KeySetStore와 MinHeapScheduler 기반 TTL 상태 정리 구현
- 문서 일괄 삭제의 DB FK(onDelete: CASCADE) 적용 및 E2E 테스트 추가
- Lint, Build, Unit, E2E로 분리된 CI 품질 게이트 구성
- 1GB RAM 서버 환경을 고려한 Docker 배포 설정 정리

5-1. Synapse · LLM 흐름 제어

Problem LLM 응답은 확률적입니다. 질문을 생성할 때마다 표현과 판단이 달라질 수 있고, 서버가 요구한 종료 신호가 항상 기대한 시점에 오지 않을 수 있습니다. 면접 서비스에서는 이 문제가 단순 품질 문제가 아니라 “면접이 진행되고 있다”는 경험 자체의 신뢰 문제로 이어집니다. Decision 저는 LLM이 결정한 값을 그대로 서비스 상태로 쓰지 않고, 서버 정책이 최종 판단을 덮어쓸 수 있게 분리했습니다. 종료 조건, 질문 중복 여부, 응답 형식 검증은 서버 레이어에서 다시 판단하도록 했습니다.	구현 단위 - 질문 흐름 제어 정책 - 질문 이력 기반 중복 방지 - 재시도 실패 시 종료 흐름 처리 - 응답 형식 검증 - 사용자 문서 컨텍스트 반영
--	---

결과적으로 정리한 기준

LLM 기반 기능에서 중요한 것은 더 긴 프롬프트만이 아니었습니다. 실패했을 때 서버가 어디서 멈추고, 어떤 값을 우선하고, 어떤 조건을 테스트로 확인할지가 더 중요했습니다. 이 판단을 통해 AI 응답의 변동성을 서비스 정책 안에서 다루는 구조를 만들었습니다.



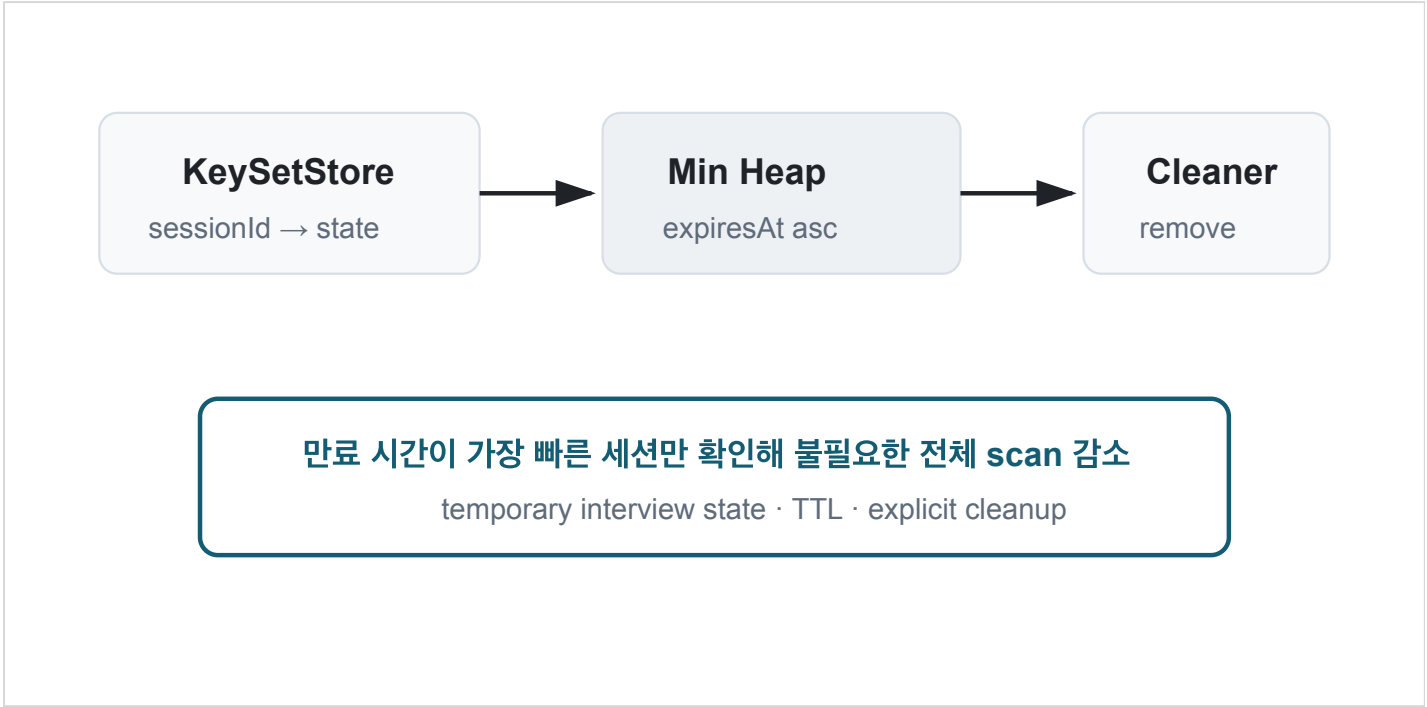
5-2. Synapse · 세션 상태와 삭제 정합성

런타임 상태 저장

면접 중 필요한 질문 이력과 방문 주제는 영속 데이터가 아니라 만료되어야 하는 런타임 상태로 보았습니다. Redis, DB, In-Memory 구조를 비교한 뒤 최소 상태 기반 In-Memory 구조를 선택했고, KeySetStore와 MinHeapScheduler로 TTL 정리 흐름을 구현했습니다.

문서 일괄 삭제

사용자 문서처럼 연관 엔티티가 많은 데이터 삭제에서 애플리케이션 반복 삭제는 누락과 성능 저하 위험이 있었습니다. DB FK(onDelete: CASCADE)에 삭제를 위임하고, 요청 DTO 검증과 E2E 테스트를 함께 추가했습니다.



운영 보강

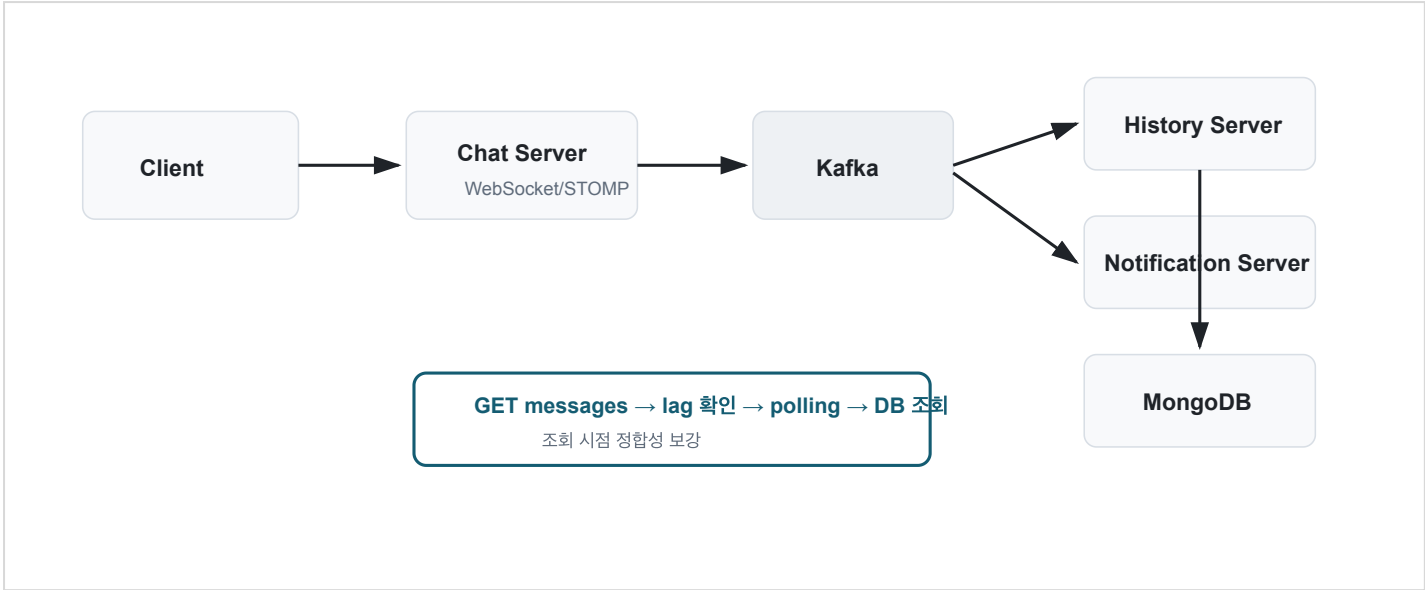
- Auth/User/Document E2E 테스트를 보강했습니다.
- MinHeapScheduler와 KeySetStore 단위 테스트를 추가했습니다.
- CI를 Lint, Build, Unit, E2E 단계로 분리했습니다.
- Docker 실행 산출물 포함 범위를 정리해 제한된 서버 환경을 고려했습니다.

5-3. SmileTogether · 프로젝트 개요

실시간 메신저 플랫폼 · 2025.01 ~ 2025.03 · Java, Spring Boot, Kafka, MongoDB, FCM, Docker Compose

역할 Team Lead & Backend Developer	핵심 문제 비동기 저장 지연과 조회 정합성	주요 구현 KafkaLagChecker, FCM 알림 서버 MVP
--	-----------------------------------	--

SmileTogether는 채팅 서버, 히스토리 서버, 알림 서버가 분리된 실시간 메신저 플랫폼입니다. 메시지는 Kafka를 통해 비동기로 저장되기 때문에, 사용자가 전송 직후 히스토리를 조회하면 아직 저장되지 않은 메시지가 누락처럼 보일 수 있었습니다.



제가 맡은 일

- latest offset과 committed offset을 비교해 직후 조회 누락 가능성을 낮추는 KafkaLagChecker 구현
- lag이 남아 있을 때 최대 5회, 100ms 간격으로 polling 후 MongoDB 조회
- 알림 서버 MVP 구성과 FCM 구독 정보 저장 API 구현
- Kafka consumer, FCM 전송 DTO와 서비스 흐름 구현
- 서버 간 통신과 CORS 문제 해결
- Docker Compose 환경 변수 분리

5-4. SmileTogether · 정합성과 책임 분리

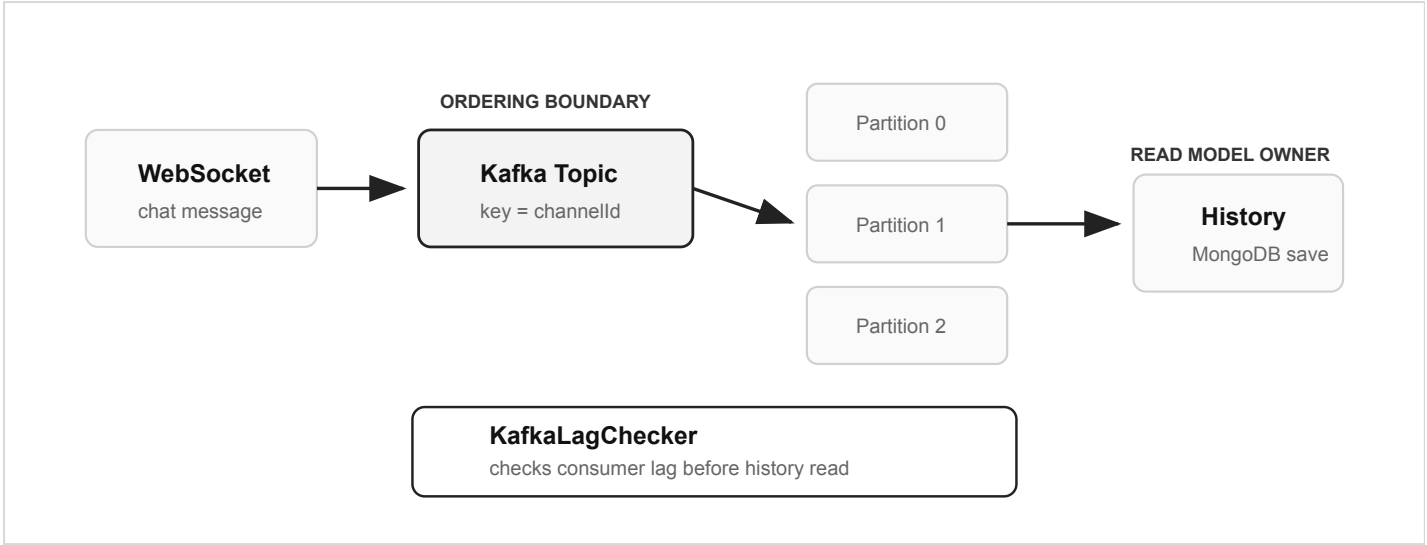
저장 지연이 누락처럼 보이는 구간 줄이기

사용자가 메시지를 보낸 직후 히스토리를 조회했을 때, Kafka consumer가 아직 MongoDB에 저장하지 않았다면 사용자는 메시지가 사라졌다고 느낄 수 있습니다. 저는 이 문제를 데이터 유실이 아니라 사용자 체감 정합성 문제로 보았습니다.

History API 조회 전에 offset lag을 확인하고, lag이 남아 있으면 짧은 polling 이후 DB를 조회하도록 구성했습니다. 이 방식은 저장 경로를 동기화로 바꾸지 않으면서도 직후 조회 누락 가능성을 낮추는 완화책이었습니다. 특정 메시지의 저장 완료 보장으로 확대하지 않았습니다.

조회 경로 보강

- latest offset 확인
- committed offset 확인
- lag 존재 시 100ms 간격 polling
- 최대 5회 확인 후 MongoDB 조회



알림 서버 분리

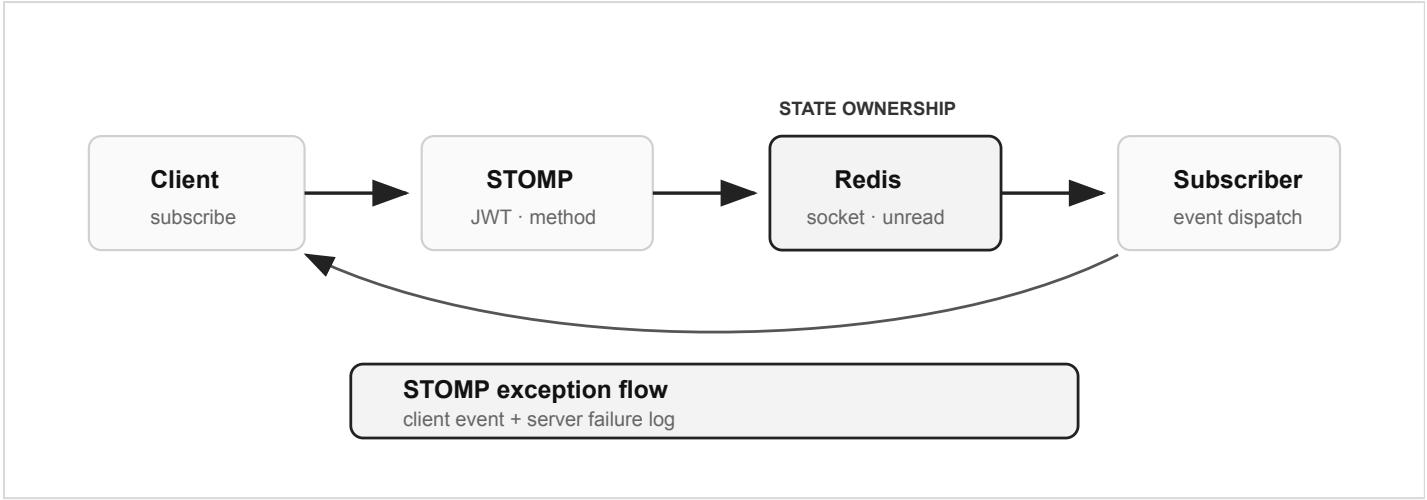
채팅 서버가 알림 전송까지 직접 가지지 않도록 알림 서버 MVP를 구성했습니다. FCM 구독 정보를 저장하고, Kafka consumer를 통해 알림 전송 흐름을 처리하게 만들었습니다. 이후 하드코딩된 대상이 아니라 채널 멤버 기준으로 알림을 보낼 수 있게 수정했습니다.

5-5. FaceFriend · 실시간 상태 관리

실시간 채팅 기반 서비스 · 2024.03 ~ 2024.06 · Java, Spring Boot, WebSocket, STOMP, Redis, MySQL

역할 Backend Developer	핵심 문제 접속 상태와 채팅방 상태 분리	주요 구현 Redis 상태 저장, STOMP 오류 흐름
--------------------------------	----------------------------------	--

FaceFriend에서는 WebSocket/STOMP 기반 채팅을 구현하며 접속 상태, 채팅방 입장 상태, 미수신 메시지 복구처럼 빠르게 변하는 상태를 다뤘습니다. 메시지는 영속 데이터로 관리해야 하지만, 사용자가 지금 접속 중인지와 같은 현재성 정보는 빠르게 바뀌는 런타임 상태입니다.



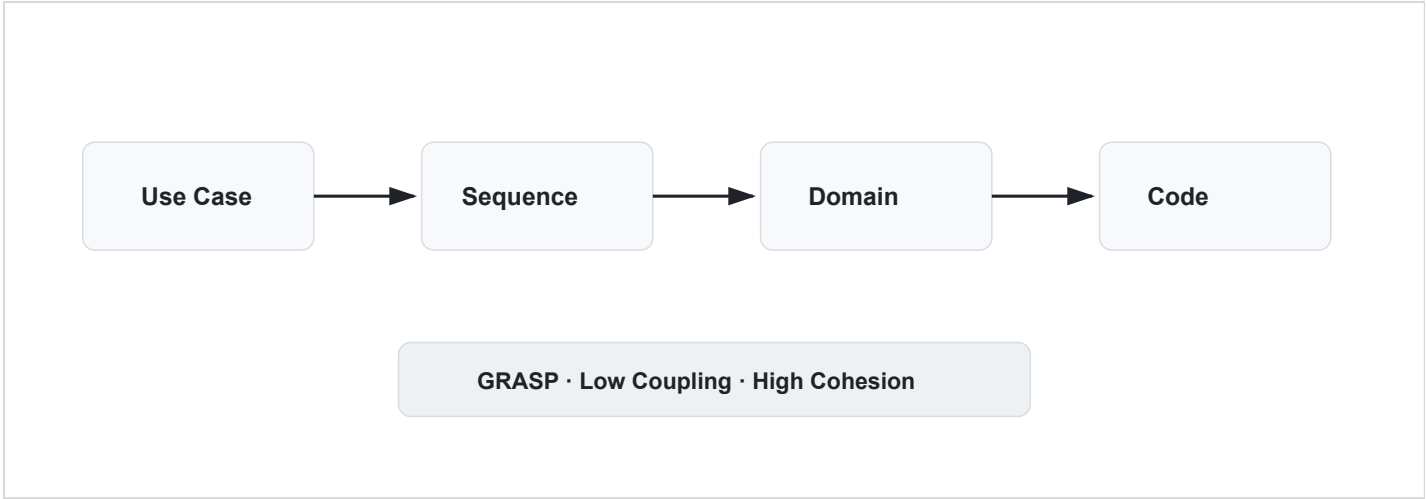
구현과 판단

- SocketInfo와 ChatRoomInfo를 Redis에 저장해 접속 상태와 채팅방 상태를 구분했습니다.
- STOMP 통신 중 오류가 발생했을 때 프론트엔드가 실패를 인지할 수 있도록 오류 응답 흐름을 정리했습니다.
- 채팅방 퇴장, 미수신 메시지 복구, 이미지 마스킹 수준 변경 등 상태 변경 흐름을 보강했습니다.
- AOP와 서비스 계층의 책임을 조정해 채팅 관련 부가 처리와 핵심 로직이 섞이지 않도록 정리했습니다.

5-6. KiKi Kiosk · 객체지향 설계 경험

객체지향 기반 키오스크 시스템 · 2024.03 ~ 2024.06 · Java, Spring Boot, MySQL

KiKi Kiosk는 키오스크 주문에 어려움을 느끼는 정보 취약계층을 위해 주문 단계를 하나씩 나눠 점원과 대화하듯 주문할 수 있게 만든 서비스입니다. 객체지향 분석 및 설계 수업의 산출물을 바탕으로 Use Case, Sequence Diagram, Class Diagram을 작성하고 구현에 반영했습니다.



구현과 설계

- 주문, 결제, 메뉴, 정산 도메인을 분리하고 객체별 책임과 상태를 정의했습니다.
- Controller, Service, Repository 계층으로 책임을 나누고 관리자 로그인/로그아웃 기능을 구현했습니다.
- WebMvcConfigurer를 통한 글로벌 CORS 정책과 전역 예외 처리 규칙을 구현했습니다.
- GRASP 원칙과 Controller, Creator, Expert, DTO 패턴을 적용해 Low Coupling, High Cohesion 구조를 지향했습니다.

이 프로젝트는 복잡한 인프라보다 설계 산출물에서 실제 코드 책임을 어떻게 도출하는지에 집중한 경험입니다. 기능 구현 전에 객체 책임을 먼저 나누는 훈련이 이후 백엔드 계층 설계에도 도움이 되었습니다.

5-7. 보조 프로젝트와 활동

Relanz

스트레스 관리 및 챌린지 플랫폼 · Python, Django, JavaScript, SQLite

- 이메일 중복 확인 View와 Fetch API 연동을 추가했습니다.
- 세션 의존 데이터 전달을 함수 기반 조회로 리팩토링했습니다.
- 비밀번호 표시/숨김, 입력 검증 상태, 반응형 레이아웃을 보강했습니다.
- Migration 충돌을 정리하고 태그 데이터 표시 규격을 맞췄습니다.

Java GC Puzzle

Java 메모리/GC 학습용 웹 사이트 · Personal Project

Java 메모리 구조와 가비지 컬렉션 동작 원리를 퍼즐 게임 형태로 학습할 수 있도록 만든 개인 프로젝트입니다. 객체 생명주기와 GC 흐름을 시각적으로 정리하며 구현과 개념을 연결해 이해하려고 했습니다.

우아한테크코스 프리코스

문자열 계산기, 자동차 경주, 로또 미션을 구현하며 파싱, 검증, 계산, 전략 객체를 분리했습니다. 코드 리뷰를 통해 DTO 책임, MVC 역할, 인터페이스 추상화에 대해 의견을 나눴습니다.

오픈소스 기여 모임 10기

Spring Session 공식 문서에서 MongoDB와 Hazelcast 커뮤니티 익스텐션 항목이 누락된 것을 확인하고 수정 제안을 제출했습니다. 기존 코드와 문서의 맥락을 먼저 읽는 협업 방식을 익혔습니다.

활동 경험

멋쟁이사자처럼 국민대 대표로 약 40명 규모의 동아리 운영과 해커톤 진행을 맡았고, Django와 AWS 세션을 진행했습니다. 이 경험은 구현뿐 아니라 팀원이 같은 목표와 속도로 움직일 수 있게 설명하고 정리하는 역량을 키우는 계기가 됐습니다.

6. 기술 및 사용 툴 / 7. 성과 및 강점 요약

기술 및 사용 툴 Backend: Java, Spring Boot, NestJS, Node.js, TypeScript Database: MySQL, PostgreSQL, MongoDB, JPA, TypeORM Realtime / Messaging: Kafka, Redis, WebSocket, STOMP Infra: Docker, Docker Compose, GitHub Actions, AWS, NCP, Nginx Test: Unit Test, E2E Test, CI 품질 게이트	논문 및 수상 - JWT 기반 사용자 인증 및 재발급 구조 연구 - 한국정보과학회 학술발표논문집 게재 2024 한국컴퓨터종합학술대회 학부생/주니어 논문경진대회 장려상 - 멋쟁이사자처럼 X 국민대 해커톤 금상
자격 정보처리기사 한국산업인력공단 · 취득 2025.09.12	학력 국민대학교 글로벌인문·지역대학 한국역사학과 (문학사) 소프트웨어전공 (공학사, 컴퓨터공학심화) 졸업 2025.02.19
수료 및 활동 - 네이버 부스트캠프 웹·모바일 10기 - 오픈소스 기여 모임 10기 - 우아한테크코스 6기 프리코스 - 멋쟁이사자처럼 국민대 대표	

성과 및 강점 요약

상태 관리, 비동기 정합성, 실시간 통신, 테스트 기반 검증을 실제 프로젝트에서 반복적으로 다뤘습니다. 기업에서는 기능 구현뿐 아니라 장애로 이어질 수 있는 실패 조건을 먼저 정의하고, 그 조건을 코드와 테스트에 반영하는 방식으로 기여할 수 있습니다.

8. 마무리 소개

저는 새로운 기술을 빠르게 나열하기보다, 맡은 도메인에서 실패 조건을 먼저 정의하고 그 조건을 코드와 테스트에 반영하는 방식으로 기여하고 싶습니다. 백엔드 시스템에서는 데이터 정합성, 비동기 처리 지연, 세션 상태 만료, 배포 환경 차이가 사용자 경험으로 바로 드러난다고 생각합니다.

제가 해온 프로젝트들은 모두 이 문제를 작게나마 다룬 경험이었습니다. AI 면접 서비스에서는 LLM 응답을 서버 정책 안에서 제어했고, 실시간 메신저에서는 Kafka 저장 지연이 조회 경험을 해치지 않도록 보강했으며, WebSocket 채팅에서는 현재성 정보를 Redis 상태로 분리했습니다.

초기 기여

도메인 흐름과 실패 조건을 파악하고, 기존 테스트와 운영 경로를 먼저 확인하겠습니다.

구현 방식

기능 구현과 함께 검증 조건, 예외 흐름, 데이터 정합성 기준을 명확히 하겠습니다.

협업 방식

변경 이유와 대안을 문서와 코드 리뷰에서 설명해 팀원이 판단을 공유할 수 있게 하겠습니다.

성장 방향

확인 가능한 사실과 실행 가능한 개선을 기준으로 백엔드 시스템 신뢰성을 높여겠습니다.

확인되지 않은 성과를 크게 말하기보다, 직접 구현하고 검증한 사실을 바탕으로 팀이 신뢰할 수 있는 백엔드 시스템을 만드는 개발자가 되겠습니다.